

Avance de Proyecto: App de Finanzas del Hogar

ICI 324 - Bases de Datos y Programación Web

Maximiliano Felipe Rozas Rifo, Gladys Carvacho Mondaca

29 de Septiembre 2026

1. Definición del Proyecto (Contexto)

1.1 Introducción y Problema

En la mayoría de los hogares o viviendas compartidas, llevar el control de los gastos mensuales resulta complejo y desordenado. Frecuentemente, los integrantes de la vivienda intentan registrar pagos de supermercado, servicios básicos o imprevistos de manera mental o en soportes informales, lo que inevitablemente genera descuadres a fin de mes, dificulta la capacidad de ahorro y crea conflictos al no tener claridad sobre quién realizó qué pago. Esta situación afecta especialmente a familias, parejas o estudiantes universitarios que comparten vivienda y necesitan transparentar, ordenar y administrar sus finanzas de manera conjunta.

1.2 Afectados

Cualquier grupo familiar, parejas o estudiantes que compartan vivienda y necesiten transparentar, ordenar y administrar sus finanzas de manera conjunta.

1.3 Objetivo de Negocio

Para abordar esta problemática, el objetivo central de este proyecto es centralizar el registro de ingresos y egresos del hogar para proporcionar visibilidad en tiempo real de la situación financiera. Para lograrlo, se desarrollará una aplicación web responsiva que permitirá a los miembros registrar sus transacciones diarias desde cualquier dispositivo. La plataforma categorizará automáticamente los movimientos y entregará reportes filtrables, facilitando la auditoría de gastos y la optimización del presupuesto mensual familiar. Todo este flujo de información estará respaldado por protocolos de seguridad que garanticen la protección de los datos personales y financieros de los usuarios.

1.4 Propuesta de Solución

Desarrollar una aplicación web responsiva que permita a los miembros del hogar registrar sus transacciones diarias (ingresos y gastos) desde cualquier dispositivo. La plataforma categorizará automáticamente los movimientos y entregará reportes filtrables para facilitar el análisis financiero del hogar.

1.5 Tecnologías Utilizadas

El desarrollo del sistema se basará en las siguientes tecnologías para asegurar un rendimiento óptimo y cumplir con los requerimientos arquitectónicos:

- **Frontend y Backend:** El proyecto utilizará el framework Next.js (con App Router) basado en React y TypeScript, permitiendo una arquitectura robusta que integre tanto la interfaz de usuario como las rutas de la API en un solo repositorio.
- **Base de Datos:** Se implementará PostgreSQL a través de la plataforma Supabase, la cual provee un motor relacional sólido, asegurando la integridad de los datos y escalabilidad futura.
- **Control de Versiones y Despliegue:** El código fuente será alojado en GitHub, facilitando el trabajo colaborativo, y se utilizarán servicios en la nube para el despliegue continuo de la aplicación web.

2. Requerimientos del Sistema

2.1 Requerimientos Funcionales (RF)

- **RF01 (Autenticación):** El sistema debe permitir a los usuarios iniciar sesión mediante credenciales válidas.
- **RF02 (Cierre de sesión):** El sistema debe permitir a los usuarios cerrar su sesión activa.
- **RF03 (Crear Categoría):** El sistema debe permitir registrar nuevas categorías para clasificar gastos (ej. Supermercado, Arriendo).
- **RF04 (Modificar Categoría):** El sistema debe permitir editar el nombre o descripción de una categoría existente.
- **RF05 (Listar Categorías):** El sistema debe mostrar un listado con todas las categorías activas.
- **RF06 (Eliminar Categoría):** El sistema debe permitir la eliminación (lógica o física) de categorías sin transacciones asociadas.
- **RF07 (Registrar Miembro):** El sistema debe permitir dar de alta a un nuevo integrante del hogar.
- **RF08 (Modificar Miembro):** El sistema debe permitir actualizar los datos de perfil de los integrantes del hogar.
- **RF09 (Desactivar Miembro):** El sistema debe permitir cambiar el estado de un miembro a "inactivo" sin borrar su historial financiero.
- **RF10 (Registrar Transacción):** El sistema debe permitir registrar un movimiento de dinero, vinculando obligatoriamente a un Miembro y una Categoría.
- **RF11 (Filtros Obligatorios):** El sistema debe permitir consultar el historial de transacciones filtrando por un rango de fechas y el tipo de movimiento (Ingreso o Egreso).
- **RF12 (Filtros Opcionales):** El sistema debe permitir acotar la búsqueda del historial mediante filtros opcionales: miembro asociado, categoría, rango de monto y medio de pago.
- **RF13 (Autenticación y Biometría):** Se ampliará el inicio de sesión con credenciales para exigir un bloqueo por huella dactilar o biometría nativa del dispositivo cada vez que se abra la app, protegiendo los historiales de ingresos y egresos.

2.2 Requerimientos No Funcionales (RNF)

- **RNF01 (Usabilidad):** La interfaz de usuario debe ser Mobile-First, garantizando una visualización correcta y fluida en dispositivos móviles.
- **RNF02 (Seguridad y Privacidad de Datos):** Las contraseñas deben almacenarse encriptadas mediante algoritmos de hash (ej. bcrypt). Adicionalmente, los datos sensibles (como el correo electrónico y el nombre del usuario) deben estar ofuscados en la interfaz para sesiones no privilegiadas y transmitirse siempre bajo protocolos seguros (HTTPS) para evitar la exposición de la identidad.
- **RNF03 (Disponibilidad):** El backend de la aplicación debe estar alojado en un servicio en la nube para garantizar acceso continuo.
- **RNF04 (Control de Versiones):** El código fuente debe mantenerse sincronizado y versionado en una estructura de carpetas dentro de un repositorio de GitHub.
- **RNF05 (Persistencia y Escalabilidad):** El sistema debe almacenar los datos en un motor relacional en la Etapa 1, con arquitectura preparada para migrar a NoSQL en la Etapa 2.
- **RNF06 (Rendimiento):** El sistema debe procesar y mostrar los resultados del historial financiero en un tiempo de respuesta menor a 2 segundos.
- **RNF07 (Compatibilidad):** La plataforma debe renderizarse correctamente en navegadores web modernos (Chrome, Safari, Firefox).
- **RNF08 (Aislamiento y Autorización):** El sistema debe aplicar validaciones estrictas a nivel de backend para garantizar que un usuario autenticado solo tenga acceso a consultar y modificar los datos financieros de su propio grupo familiar.
- **RNF09 (Mantenibilidad):** El código fuente debe aplicar separación de responsabilidades para facilitar la conexión a la base de datos y la invocación de consultas.

3. Casos de Uso y Prototipos

3.1 Diagrama General de Casos de Uso

El siguiente diagrama detalla los actores principales del sistema y su interacción con las funcionalidades base.

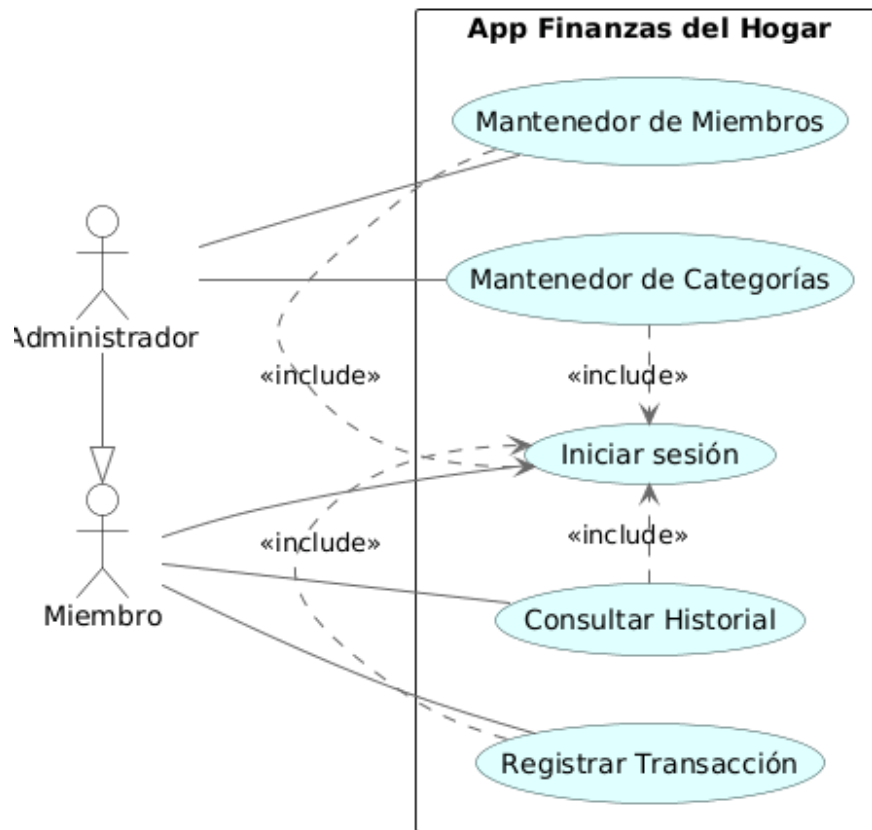


Figura 1: Diagrama General de Casos de Uso

3.1.1 Descripción de Casos de Uso Principales

CU01: Registrar Transacción

- **Actor Principal:** Miembro.
- **Precondiciones:** El usuario debe tener una sesión activa en el sistema.
- **Flujo Principal:**
 1. El usuario accede a la interfaz principal y selecciona "Nueva transacción".
 2. Selecciona el tipo de movimiento (Ingreso o Gasto).
 3. Ingresa el monto numérico y selecciona una categoría asociada.
 4. (Opcional) Selecciona el medio de pago y añade una descripción.
 5. Presiona el botón "Guardar".
 6. El backend valida los datos ingresados y la sesión del usuario.
 7. El sistema inserta el registro en la base de datos y retorna un mensaje de éxito.

- **Postcondiciones:** La transacción queda almacenada, vinculada al miembro, y el balance mensual se actualiza.

CU02: Consultar Historial

- **Actor Principal:** Miembro.
- **Precondiciones:** El usuario debe tener una sesión activa en el sistema.
- **Flujo Principal:**
 1. El usuario navega a la sección "Historial".
 2. El sistema carga por defecto todos los movimientos financieros del mes en curso.
 3. El usuario aplica filtros según sus necesidades (rango de fechas, ingresos o egresos).
 4. El sistema ejecuta la consulta a la base de datos considerando los parámetros ingresados.
 5. La interfaz renderiza la lista de transacciones filtradas y calcula los nuevos totales.
- **Postcondiciones:** El usuario visualiza la información financiera acotada a sus criterios de búsqueda.

3.2 Diagramas de Secuencia

Caso Transaccional: Registrar Gasto

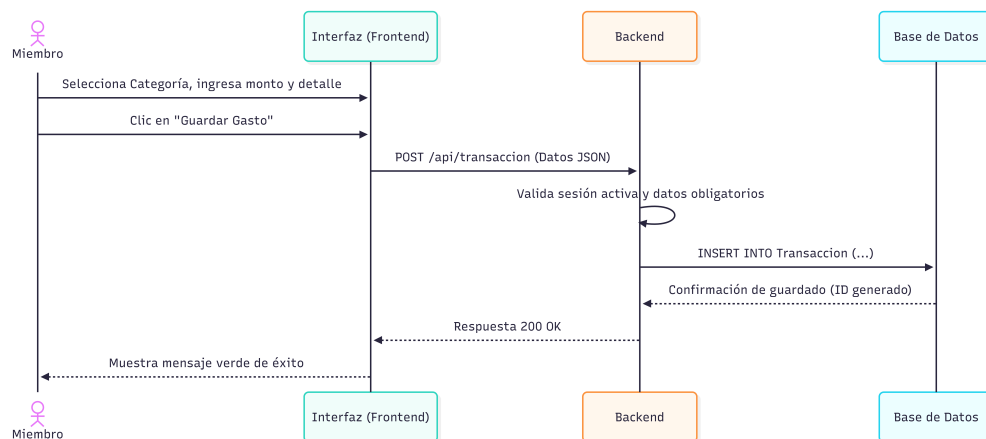


Figura 2: Flujo de Secuencia para registro de una transacción

Caso Consulta: Historial de Movimientos

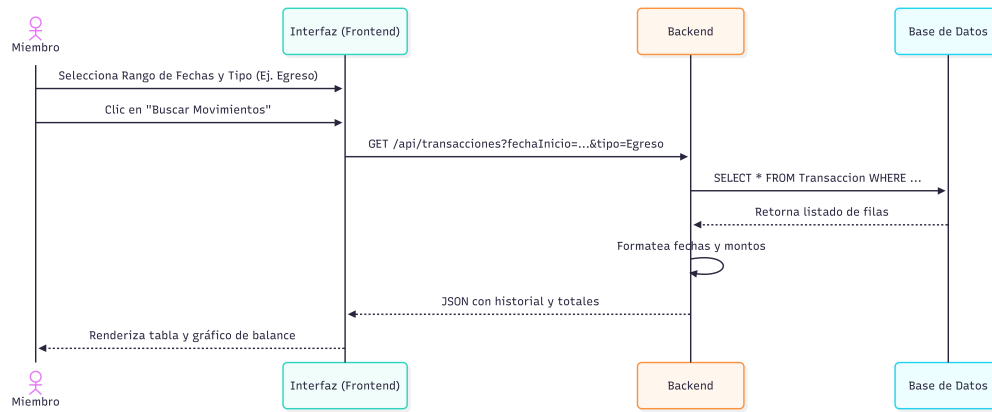


Figura 3: Flujo de Secuencia para consulta con filtros

3.3 Mockups / Prototipos de Pantallas

Justificación de diseño: Las interfaces fueron diseñadas siguiendo un enfoque Mobile-First, priorizando formularios limpios y botones amplios para facilitar la carga rápida de gastos desde un teléfono celular.

9:41

Cancelar **Nueva transacción** Guardar

- Gasto + Ingreso

MONTO

\$45.000

MR Maximiliano Rozas >

Fecha **Hoy, 31 ago** >

CATEGORÍA

Supermercado Arriendo Cuentas

Mascotas Salud Otros

MEDIO DE PAGO

Débito Efectivo Crédito

NOTA (OPCIONAL)

Compra mensual supermercado

Figura 4: Prototipo de Interfaz: Registrar Gasto

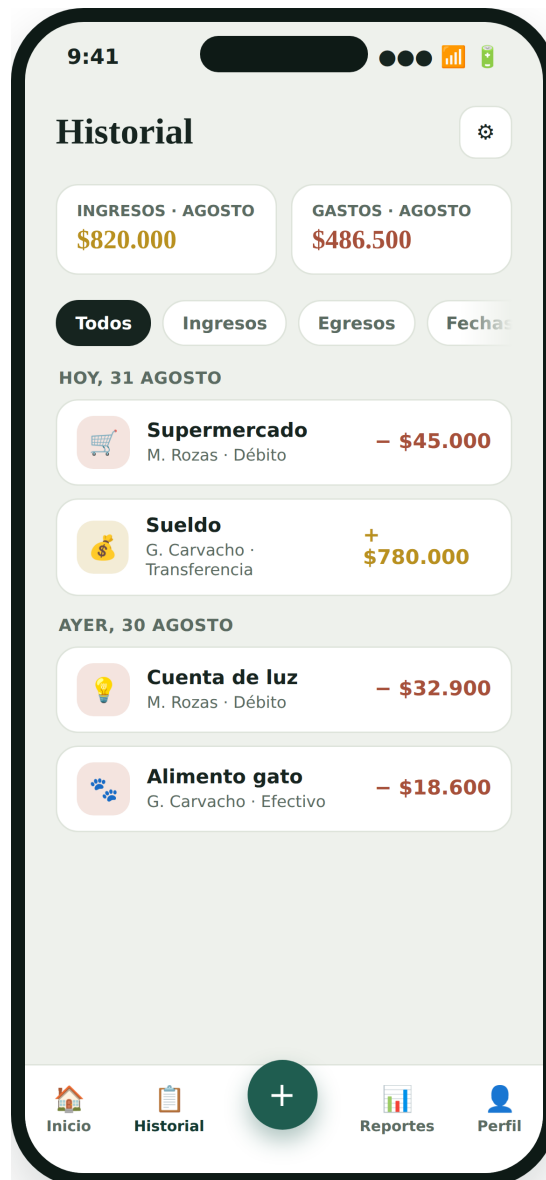


Figura 5: Prototipo de Interfaz: Consulta de Historial

4. Modelo de Datos

4.1 Supuestos y Reglas de Negocio

- Todo movimiento financiero (transacción) debe estar obligatoriamente asociado a un único miembro activo y a una sola categoría principal.
- Los montos se manejan en moneda local como valores enteros numéricos.
- Se aplicará integridad referencial estricta: no se puede eliminar un miembro o una categoría si existen transacciones históricas asociadas a ellos (se usarán estados lógicos activo/inactivo).

4.2 Modelo Relacional

El siguiente diagrama detalla la transformación del modelo conceptual a tablas relacionales, evidenciando los tipos de datos y las restricciones de integridad referencial mediante claves primarias (PK) y foráneas (FK).

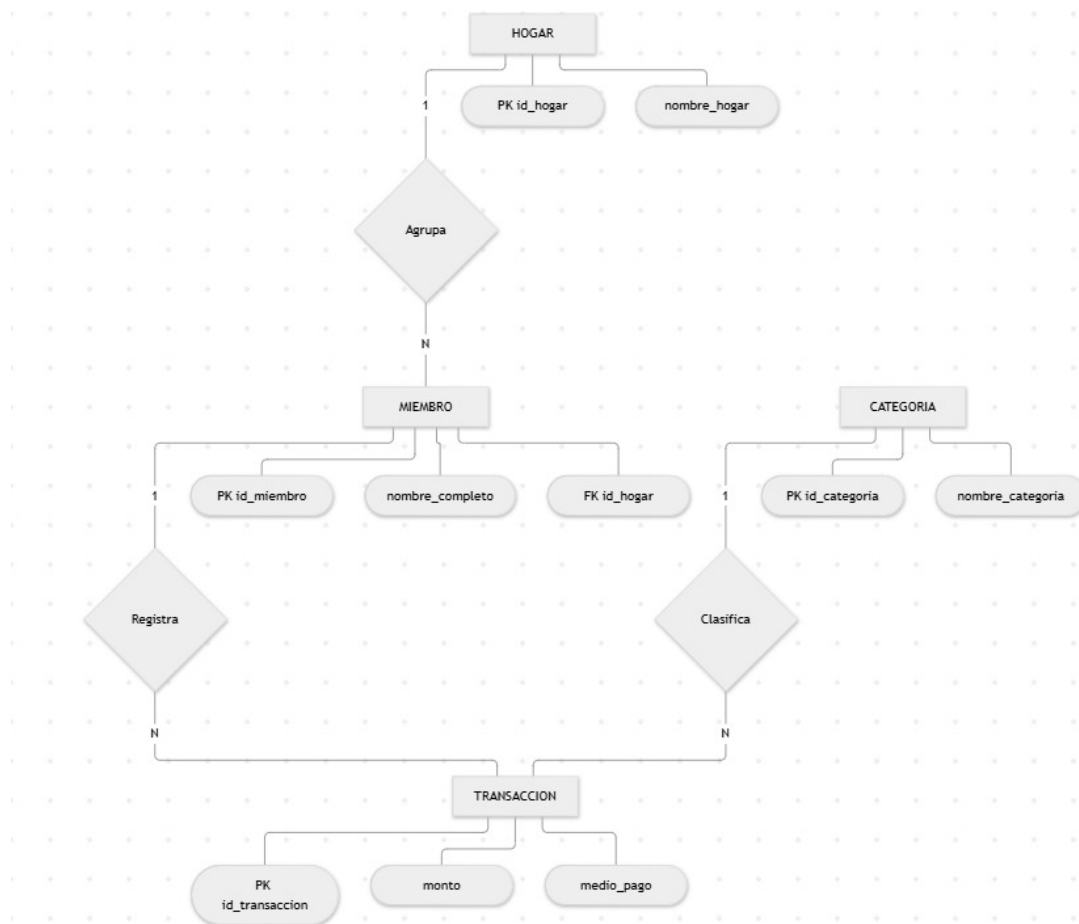


Figura 6: Diagrama del Modelo Relacional

4.3 Modelo Entidad-Relación

La siguiente figura representa las entidades, sus cardinalidades y la estructura de las tablas resultantes del modelo relacional.

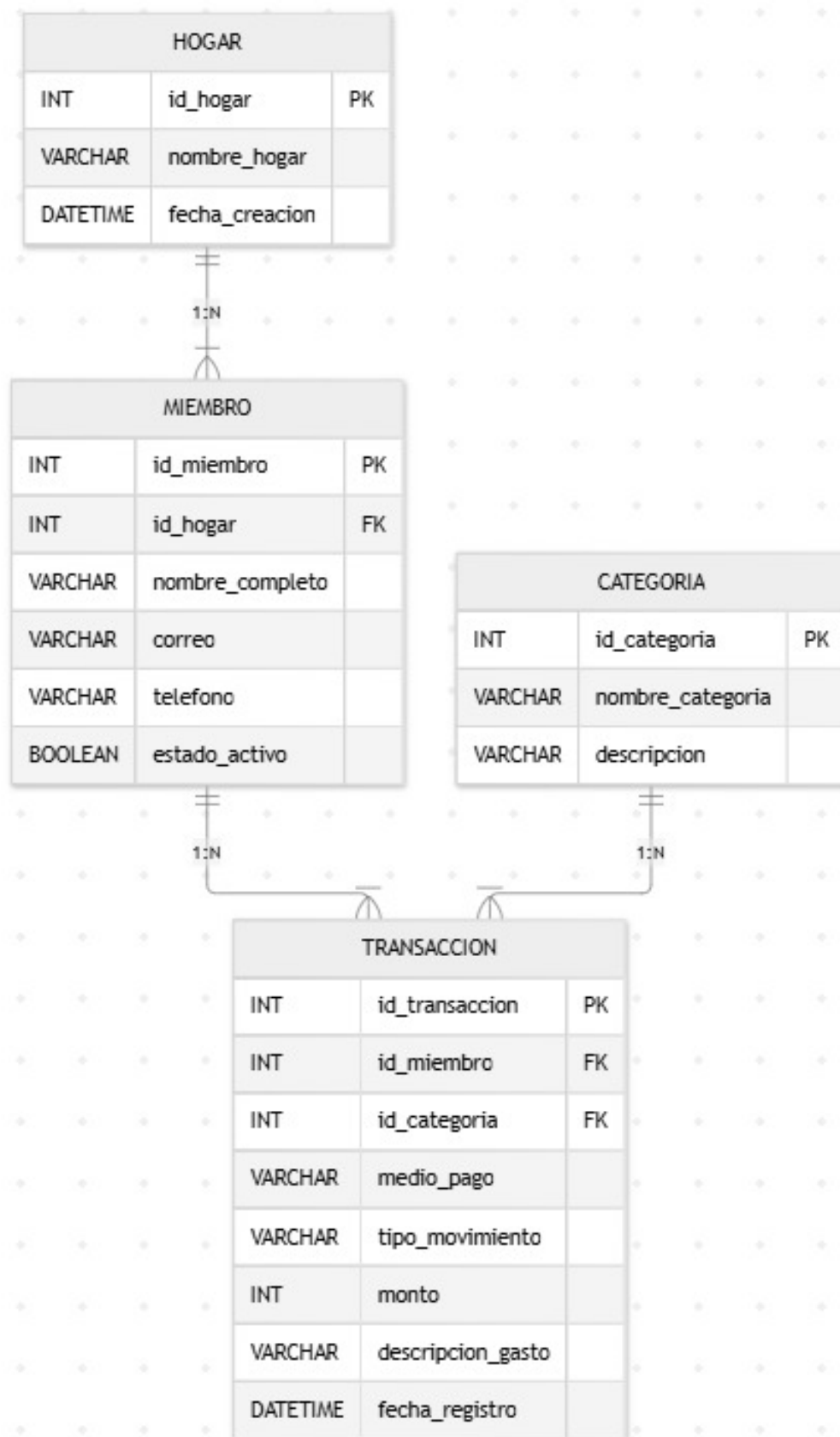


Figura 7: Modelo Entidad-Relación y Relacional

4.4 Diccionario de Datos

Tabla: Hogar

Nombre atributo	PK/FK	Tipo de dato	NULL/NOT NULL	Valores por defecto
id_hogar	PK	INT(11) AU- TO_INCREMENT	NOT NULL	0
nombre_hogar	-	VARCHAR(100)	NOT NULL	-
fecha_creacion	-	DATETIME	NOT NULL	CURRENT_TIMESTAMP

Tabla: Miembro

Nombre atributo	PK/FK	Tipo de dato	NULL/NOT NULL	Valores por defecto
id_miembro	PK	INT(11) AU- TO_INCREMENT	NOT NULL	0
id_hogar	FK	INT(11)	NOT NULL	-
nombre_completo	-	VARCHAR(100)	NOT NULL	-
correo	-	VARCHAR(100)	NOT NULL	-
estado_activo	-	BOOLEAN	NOT NULL	TRUE

Tabla: Categoria

Nombre atributo	PK/FK	Tipo de dato	NULL/NOT NULL	Valores por defecto
id_categoria	PK	INT(11) AU- TO_INCREMENT	NOT NULL	0
nombre_categoria	-	VARCHAR(50)	NOT NULL	-
descripcion	-	VARCHAR(255)	NULL	NULL

Tabla: Transaccion

Nombre atributo	PK/FK	Tipo de dato	NULL/NOT NULL	Valores por defecto
id_transaccion	PK	INT(11) AU- TO_INCREMENT	NOT NULL	0
id_miembro	FK	INT(11)	NOT NULL	-
id_categoria	FK	INT(11)	NOT NULL	-
tipo_movimiento	-	ENUM('Ingreso', 'Egre- so')	NOT NULL	'Egreso'
monto	-	INT(11)	NOT NULL	0
medio_pago	-	VARCHAR(50)	NOT NULL	'Debito'
descripcion_gasto	-	VARCHAR(255)	NULL	NULL
fecha_registro	-	DATETIME	NOT NULL	CURRENT_TIMESTAMP

5. Consultas SQL y Álgebra Relacional

Modificación de Estructura (ALTER y DROP)

1. ALTER TABLE (Agregar columna):

```
ALTER TABLE Miembro ADD COLUMN telefono VARCHAR(15) NULL;
```

2. ALTER TABLE (Modificar columna):

```
ALTER TABLE Categoria MODIFY COLUMN descripcion VARCHAR(300) NULL;
```

3. DROP TABLE:

```
DROP TABLE RespaldoTransacciones;
```

Población y Modificación de Datos (INSERT, UPDATE, DELETE)

4, 5, 6 y 7. INSERT (4 sentencias respetando integridad referencial):

```
INSERT INTO Hogar (nombre_hogar)
VALUES ('Departamento Mirador Valpo');

INSERT INTO Categoria (nombre_categoria, descripcion)
VALUES ('Supermercado', 'Compras de despensa y articulos para el hogar')
;

INSERT INTO Miembro (nombre_completo, correo, estado_activo, id_hogar)
VALUES ('Maximiliano Felipe Rozas Rifo', 'maximixasz@gmail.com', TRUE,
1);

INSERT INTO Transaccion (id_miembro, id_categoria, tipo_movimiento,
    monto, medio_pago, descripcion_gasto)
VALUES (1, 1, 'Egreso', 45000, 'Debito', 'Compra mensual supermercado');
```

8 y 9. UPDATE (2 sentencias):

```
UPDATE Miembro SET telefono = '+56912345678' WHERE id_miembro = 1;

UPDATE Transaccion SET monto = 48000 WHERE id_transaccion = 1;
```

10 y 11. DELETE (2 sentencias):

```
DELETE FROM Transaccion WHERE id_transaccion = 2;

DELETE FROM Categoria WHERE id_categoria = 99;
```

Consultas y Álgebra Relacional (SELECT)

12. SELECT 1 (Consulta Simple):

```
SELECT nombre_completo, correo
FROM Miembro
WHERE estado_activo = TRUE;
```

Álgebra Relacional:

$$\pi_{\text{nombre_completo}, \text{correo}}(\sigma_{\text{estado_activo}=\text{TRUE}}(\text{Miembro}))$$

13. SELECT 2 (Consulta con 1 JOIN): *Obtener los miembros activos y el nombre del hogar al que pertenecen.*

```
SELECT H.nombre_hogar, M.nombre_completo
FROM Miembro M
JOIN Hogar H ON M.id_hogar = H.id_hogar
WHERE M.estado_activo = TRUE;
```

Álgebra Relacional:

$$\pi_{\text{nombre_hogar}, \text{nombre_completo}}(\sigma_{\text{estado_activo}=\text{TRUE}}(\text{Miembro} \bowtie_{\text{M.id_hogar}=\text{H.id_hogar}} \text{Hogar}))$$

14. SELECT 3 (Consulta con 2 JOINS): *Obtener las transacciones mayores a 10.000, mostrando quién lo gastó y en qué hogar.*

```
SELECT H.nombre_hogar, M.nombre_completo, T.monto
FROM Transaccion T
JOIN Miembro M ON T.id_miembro = M.id_miembro
JOIN Hogar H ON M.id_hogar = H.id_hogar
WHERE T.monto > 10000;
```

Álgebra Relacional:

$$\pi_{\text{nombre_hogar}, \text{nombre_completo}, \text{monto}}\left(\sigma_{\text{monto}>10000}\left(\left(\text{Transaccion} \bowtie_{\text{T.id_miembro}=\text{M.id_miembro}} \text{Miembro}\right) \bowtie_{\text{M.id_hogar}=\text{H.id_hogar}} \text{Hogar}\right)\right)$$

6. Enlaces de Entrega

- Repositorio de Código (GitHub): <https://github.com/barsak34/App-finanzas-ici324>